

Join the lecture online on your dashboard.

A Brief History of NLP

1950s — The Turing Test

Alan Turing proposes the imitation game; rule-based NLP begins.

1

2

1960s–70s — ELIZA & SHRDLU

Early chatbots and symbolic parsing systems emerge.

1980s — Statistical NLP

Probabilistic models replace hand-crafted rules; Hidden Markov Models for speech.

3

4

1990s — Corpus Linguistics

Large text corpora enable data-driven approaches; TF-IDF and n-grams gain traction.

2001 — Neural Language Models

Bengio et al. introduce the first neural language model with word embeddings.

5

6

2013 —

Mikolov et al. (Google) revolutionize word representations with CBOW & Skip-gram.

Word2Vec

7

2014 — GloVe & Seq2Seq

Stanford's GloVe and encoder-decoder models for machine translation.

8

2017 — Attention & Transformers

"Attention Is All You Need" — Vaswani et al. change everything.

9

2018 — BERT & GPT

Contextual embeddings arrive; pre-training + fine-tuning becomes the standard paradigm.

1

2020s — Large Language Models

GPT-3, ChatGPT, LLaMA — generative AI goes mainstream.

0

<https://jonathan-hui.medium.com/nlp-word-embedding-glove-5e7f523999f6>

Table of Contents

Foundations

- The evolution of NLP and word representation
- Essential tokenization strategies and trade-offs

Discrete & Sparse Representations

- One-hot encoding and sparse limitations
- Bag-of-Words and TF-IDF models

Prediction-Based Embeddings

- The Distributional Hypothesis
- Word2Vec architectures and GloVe embeddings

Cross-Domain Applications

- Applying embeddings to audio, images, and DNA
- Optimization and vector visualization

The Future & Conclusion

- Introduction to contextual models (BERT/GPT)
- Summary and key takeaways

The Evolution of Meaning: A Step-by-Step Guide to Vector Embeddings

01. Tokenization: Breaking Down the Recipe



Raw text broken into tokens (words, characters, n-grams), like fundamental LEGO bricks for machine understanding.

02. One-Hot Encoding: The Sparse Era

cat	1	0	0	0	0	0	0	0
dog	0	1	0	0	0	0	0	0
fish	0	0	1	0	0	0	0	0
bird	0	0	0	1	0	0	0	0
...	.	.	.	.	.	.	.	.
etc	0	0	0	0	0	0	1	1

Format Example	Dimensionality	Semantic Meaning
Sparse (One-Hot)	[0, 0, 1, 0, 0, ...]	High (50k+)

Simplest numeric conversion; each word has a unique index, creating massive, 99% empty matrices that fail to capture relationships.

03. Bag-of-Words (BoW): Document Fingerprinting



Document "fingerprint" based on word frequency, losing critical word order and grammar.

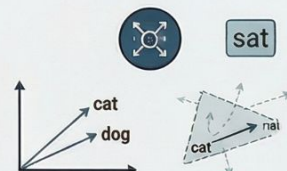
04. TF-IDF: The Smart Highlighter

the is a common word
earthquake is words
magnitude rare were
seismic whose the words shared
to the words.

Dims common words and brightens rare, meaningful ones to reward document-specific terms.

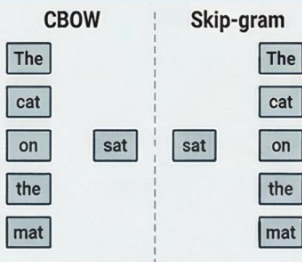
05. The Predictive Jump: Learning from Neighbors

The cat sat on the mat



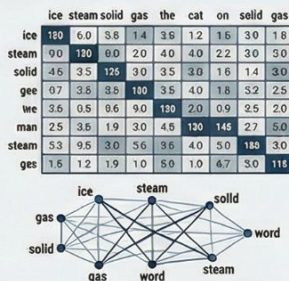
Models like Word2Vec learn word meanings by analyzing local context windows, forcing similar words to occupy nearby positions.

06. CBOW vs. Skip-gram Architecture



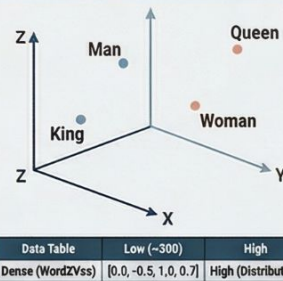
CBOW predicts a target word from surroundings; Skip-gram predicts surrounding context from a single target word.

07. GloVe: Global Co-occurrence Statistics



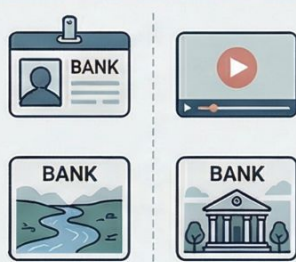
Builds a global matrix of word co-occurrences across the entire corpus, bridging count-based and prediction-based strengths.

08. Learned Geometry: King - Man + Woman = Queen



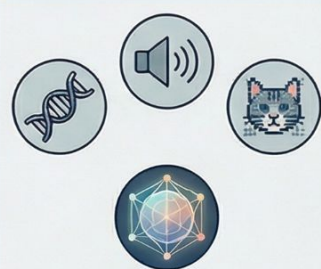
Dense vectors enable "semantic logic," where mathematical operations mirror human analogies and capture properties.

09. Contextual Embeddings: The Dynamic Shift



Modern models (BERT/GPT) generate different representations for words depending on their specific sentence context.

10. Cross-Domain Unity: Beyond Text



Need for embeddings

The Core Friction: Machines calculate, but humans symbolize

The Human Layer

Words exist as isolated tokens. Text characters do not encode inherent mathematical relationships.

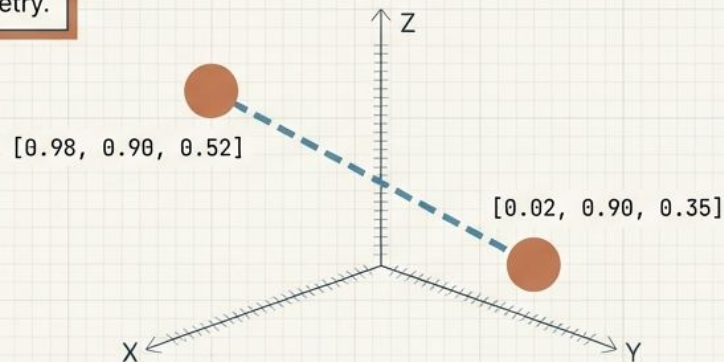
KING

QUEEN

The Goal: Transform discrete meaning into continuous geometry.

The Machine Layer

Computers require continuous mathematical spaces to perform operations and calculate distance.



The Evolution of Word Embeddings: From Discrete to Dense Representations

The field of Natural Language Processing has undergone a fundamental transformation in how it represents human language. We have moved from simple, discrete methods—which treat words as isolated, sparse entities—to sophisticated, dense vector representations that capture deep semantic meaning within high-dimensional geometric spaces.

From Sparse to Dense

Early techniques like one-hot encoding suffered from the "curse of dimensionality" and lacked semantic nuance. Modern embedding techniques represent words as continuous, fixed-size vectors, allowing models to grasp the relationships between concepts.

Capturing Semantics

By mapping words into dense vector spaces, algorithms can mathematically encode linguistic properties. This enables systems to recognize that "king" is to "man" as "queen" is to "woman," effectively capturing synonymy and relatedness.

Key Milestones

This evolution has been defined by critical breakthroughs, ranging from early statistical models like TF-IDF to predictive architectures like Word2Vec, GloVe, and eventually the contextual breakthroughs found in modern Transformer models.

Learning Objectives

By the end of this lecture, you will master key conceptual areas that form the foundation of modern word embedding techniques:



Vector Space Transformation

- Understand the **fundamental transition** from **sparse, high-dimensional representations** to **dense, low-dimensional vector spaces**.
- Focus on techniques that **capture semantic meaning** efficiently.



Word2Vec Architecture

- Analyze the core mechanics of **Word2Vec models**.
- Examine **Continuous Bag of Words (CBOW)** and **Skip-gram architectures**.
- Review effective **training methodologies**.



Cross-Domain Applications

- Explore how **embedding techniques** transcend traditional **NLP boundaries**.
- Identify innovative applications in **audio processing, genomics, and computational biology**.

The NLP Context

Natural Language Processing serves as the essential architecture connecting human communication with machine logic, transforming raw text into meaningful insights.



NLP bridges human language and machine computation.



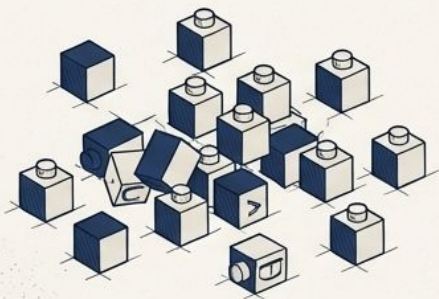
It overcomes the semantic gap between text and numerical formats.



Computational models transform linguistic input into actionable data.

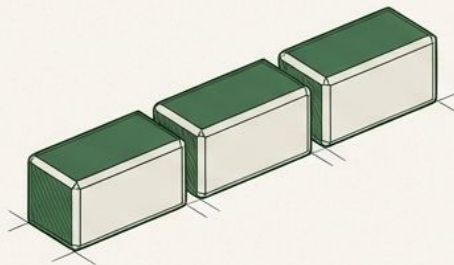
Before we can teach a machine to read, how do we decide what constitutes a single thought?

Character-Level



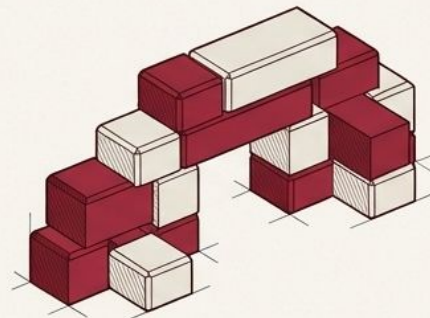
Infinite robustness.
Reconstructs partial input.
Zero inherent meaning.

Word-Level



Efficient semantic capture.
Human-readable. Requires
massive memory vocabularies.

N-gram Level



Preserves vital local context.
Causes combinatorial explosion
and data sparsity.

The Core Problem

- Computers process numerical bits, not human semantics.
- Raw text characters lack inherent meaning or context.
- Discrete symbols prevent meaningful mathematical vector operations.

1

Discrete Symbols

Words exist as **isolated tokens** without **mathematical properties** or **relationships** encoded.

2

No Inherent Math

Traditional **discrete representations** prevent meaningful **vector operations** like addition or subtraction.

3

Vector Space Mapping

Transform "**meaning**" into **geometric space** where **semantic relationships** become **mathematical operations**.

The **breakthrough solution**:

- Map **linguistic meaning** into **continuous vector space**.
- Ensure **similar words** occupy **nearby positions**.
- Enable **algebraic operations** on **concepts**.

What to embed: Tokens

Before We Begin: What is Tokenization?

Imagine a chef reading a recipe: before cooking, you first identify each ingredient. You don't cook the whole paragraph; you break it into parts.

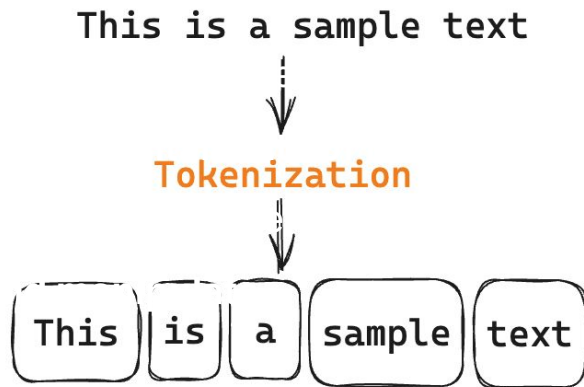
Tokenization is precisely that: breaking raw text into digestible pieces a machine can work with. Think of it like a sentence becoming a pile of **LEGO bricks** – each word or punctuation is a '**token**'. Only with these individual bricks can we start building meaning.

Tokenization: The Building Blocks

Tokenization is the essential first step for vector-based language models:

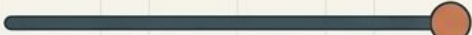
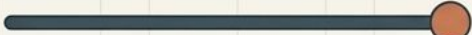
- Breaks raw text into discrete units (tokens).
- Establishes fundamental building blocks for processing.
- Determines the granularity of machine understanding.

Selecting an optimal approach depends on your specific application, dataset, and computational resources.



Defining the atomic units of meaning

Tokenization breaks raw text into digestible pieces. The chosen method determines the fundamental building blocks your model can learn.

Character-Level	Word-Level	n-gram
[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [e] [n] [i] [z] [i] [n] [g] [.]	[This] [is] [tokenizing] [.]	[Not] + [Good] → [Not Good]
 Low Semantic Granularity	 Low Semantic Granularity High	 Low Semantic Granularity High
 Low Vocabulary Size	 Low Vocabulary Size High	 Low Vocabulary Size Extreme
 Low Computational Cost High	 Low Medium High	 Low Computational Cost High
Best for noisy text & spell-checking.	Best for clean, standard NLP.	Preserves local context and negation.

Character-Level Tokenization

How It Works

- Treats each **individual letter or character** as a unique token in the vocabulary.
- Example: "Embeddings" becomes [E, m, b, e, d, d, i, n, g, s].

Strengths

- **Robustness:** Highly resilient against **misspellings, typos,** and **out-of-vocabulary words**.
- **Flexibility:** Can reconstruct meaning from **partial or corrupted input**.
- **Efficiency:** Minimal **vocabulary size** requirements.

Weaknesses

- **Computational Cost:** Significantly increases **sequence length**, demanding higher **computational resources**.
- **Semantic Granularity:** Struggles to capture **higher-level semantic concepts** efficiently.
- **Training Complexity:** Training is more challenging due to **longer dependencies**.

Word-Level Tokenization

- **Semantic Efficiency:** Captures distinct semantic concepts as individual units.
- **Classical Standard:** Serves as the primary method for count-based models.
- **Vocabulary Management:** Requires handling for rare words and unknown tokens.

Efficient Semantic Capture

Each **word maps directly** to a meaningful concept, enabling direct **semantic interpretation** of vector positions.

Vocabulary Management

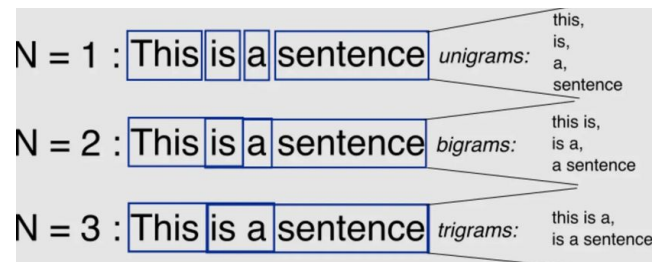
Requires **careful handling** of rare words and **unknown tokens** through techniques like **minimum frequency thresholds**.

Model Compatibility

Works seamlessly with **traditional NLP architectures**, including **count-based** and **predictive embedding methods**.

n-gram Tokenization

- Groups n consecutive words into a single composite token.
- Preserves local context often lost in word-level tokenization.
- Captures multi-word concepts like "San Francisco" as single units.



Unigrams (n=1)

Contextual limitation: Loses city-specific semantic meaning entirely.

Bigrams (n=2)

Contextual strength: Successfully preserves "San Francisco" as a singular **unit of meaning**.

Trigrams (n=3)

Contextual strength: Captures **broader semantic context** for improved phrase interpretation.



n-gram tokenization facilitates critical **semantic distinctions**, such as differentiating "Not Good" from "Good". By treating **negation phrases** as unified tokens, it prevents **opposite meanings** from collapsing into identical **vector space positions**.

Tokenization Trade-offs: Character-Level

Strengths

- Eliminates out-of-vocabulary (OOV) issues by using a tiny, fixed character set.
- Naturally captures complex linguistic morphology like prefixes and suffixes.

Drawbacks

- Increases sequence length significantly, raising computational processing requirements.

Best used for: **morphologically rich languages**, spell-checking, and handling noisy text.

Tokenization Trade-offs: Word-Level

Strengths

- Encodes distinct concepts as single units.
- Provides intuitive, human-readable tokens.
- Each token carries full semantic meaning.

Drawbacks

- Struggles with rare or unknown words.
- Requires massive, memory-heavy vocabularies.
- Fails to capture morphological relationships.

Best used for: **standard NLP tasks**, sentiment analysis, and text classification.

Tokenization Trade-offs: n-gram

This sophisticated approach groups n consecutive words into a single composite token, effectively preserving valuable local context. However, it introduces unique challenges related to vocabulary size and data sparsity.

Strengths

- Preserves local context and phrase meaning.
- Effectively handles negation phrases.
- Captures word co-occurrence patterns.

Drawbacks

- Leads to combinatorial vocabulary explosion.
- High sparsity in training data.
- Limited by fixed window size.

Best used for: **phrase detection**, language modeling, and capturing local word dependencies.

Static Embeddings

Word Embeddings

What is a Vector ?

A set of numbers that encodes the "meaning" of a word — its position in semantic space.

Why It Matters ?

Machines can't read words — but they can process numbers. Vectors bridge language and computation.

The Big Idea

Words used in similar contexts have similar vectors. Context reveals meaning.

Feature Vectors: How Words Become Numbers

Imagine describing every word along a set of dimensions — Authority, Richness, Gender, Living. Assign a score from 0 to 1 for each feature, and suddenly "King" and "Queen" are close neighbors, while "Horse" sits in a different region entirely. These numeric profiles are called feature vectors.

Word	Authority	Richness	Gender (M)	Living
King	0.95	0.90	0.98	1.00
Queen	0.95	0.90	0.02	1.00
Horse	0.10	0.20	0.60	1.00
Rocket	0.05	0.80	0.50	0.00

Intuition: Giving Words a Number

Imagine a school attendance register. Each student gets a row.

One-Hot Encoding works similarly: each word gets its own row, with its cell marked 1, and all others 0.

The problem? A 50,000-word vocabulary creates a massive matrix—over 99% empty 0s. This is the challenge of sparse representations.

The Sparse Era: Counting & Presence

- Each word occupies a unique index.
- Matrices consist primarily of zeros.
- High dimensionality leads to computational inefficiency.

Count Based Methods

One-Hot Encoding: The Motivation

Sentiment Analysis

Classifying reviews as positive or negative requires numeric data.

The Challenge

Computers cannot process raw text like "Amazing" directly.

The Solution

One-Hot Encoding converts words into unique numerical vectors.

	Idx 1	Idx 2	Idx 3	Idx 4	Idx 5	...
apple	1	0	0	0	0	0
banana	0	1	0	0	0	0
cat	0	0	1	0	0	0
dog	0	0	0	1	0	0
egg	0	0	0	0	1	0

☰ Sparse Representation: Only one active bit per vector

Core Definitions: Corpus & Vocabulary

- **Corpus:** The full collection of documents used for training.
- **Vocabulary (V):** The set of all unique words in the corpus.
- **Indexing:** Each unique word is assigned a specific numerical index.

Toy Example

Here's a small example illustrating how a corpus is processed to form a vocabulary:

Sentence 1: "I like cats"

Sentence 2: "I like dogs"

Sentence 3: "Dogs and cats are pets"

Sentence 4: "Trucks are not pets"

Resulting Vocabulary: { I, like, cats, dogs, and, are, pets, trucks, not } $\rightarrow |V| = 9$ unique words

The One-Hot Representation Recipe

To construct a one-hot vector, follow these steps:

- 1. Build the Vocabulary:** List all unique words and assign each an index (e.g., cats=0, dogs=1, trucks=2 ...)
- 2. Create a zero vector:** Make a vector of length $|V|$ filled with all zeros
- 3. Set the "1":** Place a single "1" at the index position of the target word

Example:

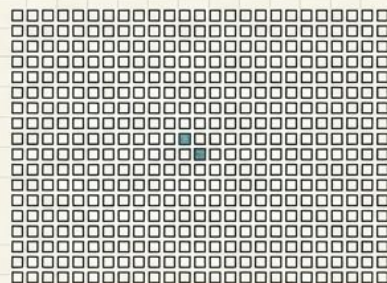
- Vocabulary size $|V| = 9$
- Word "cats" is at index 0 $\rightarrow$ vector = [1, 0, 0, 0, 0, 0, 0, 0, 0]
- Word "dogs" is at index 1 $\rightarrow$ vector = [0, 1, 0, 0, 0, 0, 0, 0, 0]
- Word "trucks" is at index 7 $\rightarrow$ vector = [0, 0, 0, 0, 0, 0, 0, 1, 0]

Key property: Each vector has exactly one "1" and all other values are "0"

The Sparse Era: Words as isolated islands

One-Hot Encoding maps each word to a unique index. While simple, it creates two catastrophic failures:

1. **Extreme Sparsity:** Wasting 99.99% of memory multiplying zeros.
2. **Semantic Blindness:** Every word is completely orthogonal.



One-Hot Representation



Dense Embedding

$$[\text{cat}] \cdot [\text{dog}] = 0$$
$$\text{Cosine Similarity} = 0$$

Mathematically, 'cat' is as different from 'dog' as it is from 'truck'.

Drawback 1: Storage & Scaling Problem



01

Scalability Issues

The vocabulary size $|V|$ can be massive in real-world applications, leading to unmanageable memory requirements.

 **Example:** Google's 1T corpus has ~13 million unique words.



02

Extreme Sparsity

Vectors are composed almost entirely of zeros. A vector of size 100,000 has only a single 1.

 **Inefficient computation:** Most matrix operations involve multiplying by zero, wasting cycles.



03

Semantic Gap

The most critical failure: distinct words are completely orthogonal. There is no encoded relationship between words.

 **Result:** No shared information between "cat" and "dog".

Drawback 2: The Similarity Problem

✚ Orthogonality Problem

Any two distinct one-hot vectors w_i and w_j are orthogonal. They have no overlapping non-zero indices.

Dot Product: $w_i \cdot w_j = 0$

Cosine Similarity: $\cos(\theta) = 0$

▒ Equidistant Vectors

The Euclidean distance between any pair of distinct words is constant.

Distance: $||w_i - w_j|| = \sqrt{2}$

Result: No cluster structure; all words are equally far apart.

SEMANTIC BLINDNESS

cat

[0, 1, 0, 0, ...]

Sim = 0



dog

[0, 0, 1, 0, ...]

cat

[0, 1, 0, 0, ...]

Sim = 0



truck

[0, 0, 0, 1, ...]

⚠ Problem:

Mathematically, "cat" is as different from "dog" as it is from "truck". The model cannot learn that cats and dogs are related animals.

From Words to Documents

"We just saw how One-Hot Encoding gives every word its own unique identity in a vector space."

"But a model can't process a list of 500 one-hot vectors for a single paragraph. We need a way to flatten that information into a single fixed-length vector."

"Bag of Words does exactly this: it takes all those one-hot vectors in a sentence and adds them together. We lose the order of the words, but we gain a 'fingerprint' of what the document is about based on word frequency."

The Bag of Words

Imagine shaking a book into a bag. All the words fall out—their order lost.

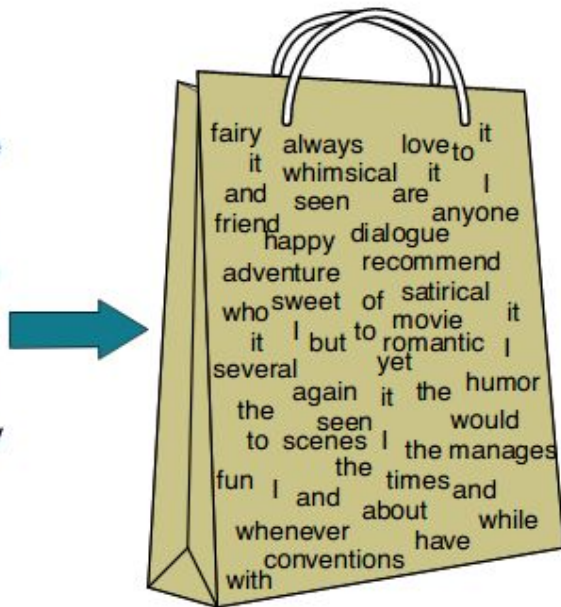
- One-Hot Encoding: Represents a single word as a high-dimensional vector. It is binary and position-independent.
- Bag of Words: Represents a complete document by counting the occurrences of every word in the vocabulary.

That's Bag of Words: **frequency matters, sequence doesn't.**



The Bag of Words

I love this movie! It's sweet,
but with satirical humor. The
dialogue is great and the
adventure scenes are fun...
It manages to be whimsical
and romantic while laughing
at the conventions of the
fairy tale genre. I would
recommend it to just about
anyone. I've seen it several
times, and I'm always happy
to see it again whenever I
have a friend who hasn't
seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

BoW in Action: Frequency Matrix Example

Sentence 1: "Students study NST"

Sentence 2: "NST study NST"

Vocabulary: { Students, study, NST } $\rightarrow |V| = 3$

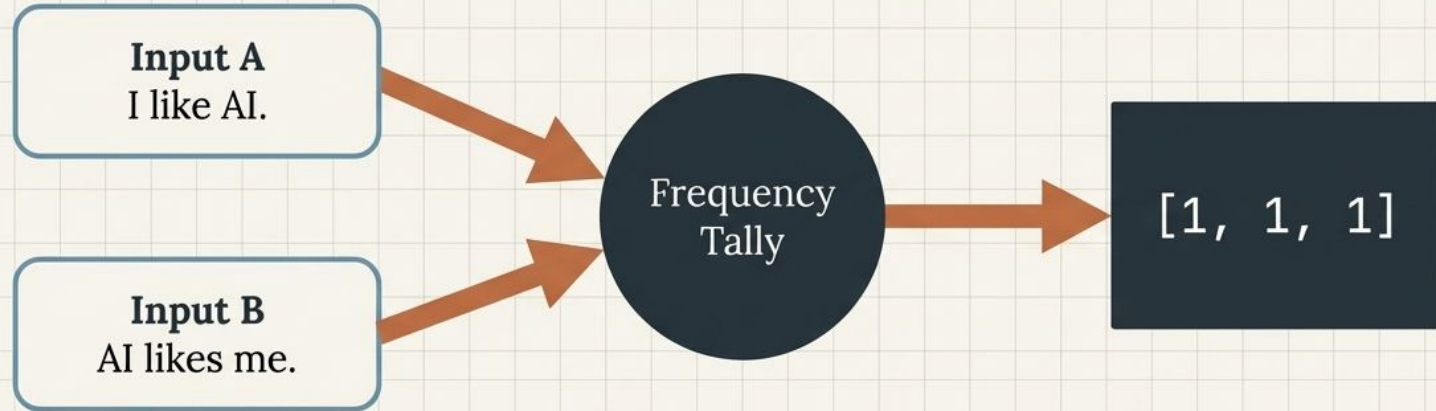
	Students	study	NST
Sentence 1: "Students study NST"	1	1	1
Sentence 2: "NST study NST"	0	1	2

Key observations:

- "NST" appears **twice** in Sentence 2 $\rightarrow$ its count is 2, not 1
- "Students" appears **zero times** in Sentence 2 $\rightarrow$ its count is 0
- Each sentence is now a **fixed-size vector**: [1,1,1] and [0,1,2]
- Vectors have the **same length** regardless of sentence length — this is the key advantage over One-Hot

Bag of Words captures frequency but destroys sequence

To represent whole documents, Bag of Words tallies up all vectors into a fixed-length signature. It successfully flattens paragraphs, but grammar, context, and negation are obliterated.



The Failure: BoW cannot distinguish between these opposite meanings because word order is ignored.

BoW: Pros & Cons

Pros

Fixed-size vectors

Consistent length for any document.

Simple & fast

Efficient computational processing.

Cons

Ignores word order

Context and grammar are lost.

No semantic meaning

Treats words as isolated units.

Sentiment Failure

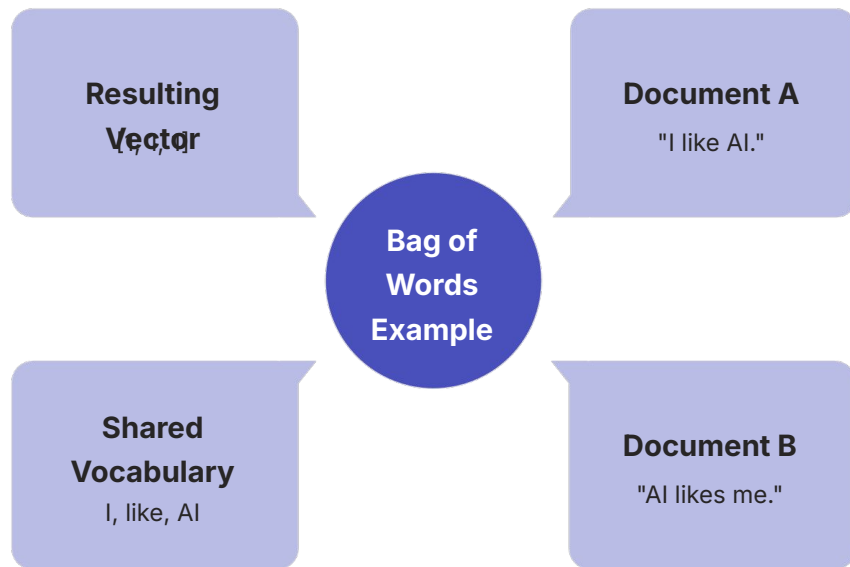
BoW fails to detect **negation** (e.g., "good" vs "not good"), resulting in nearly identical vectors that ignore critical meaning carried by word order.

BoW in Action: Example

Document A: "I like AI."

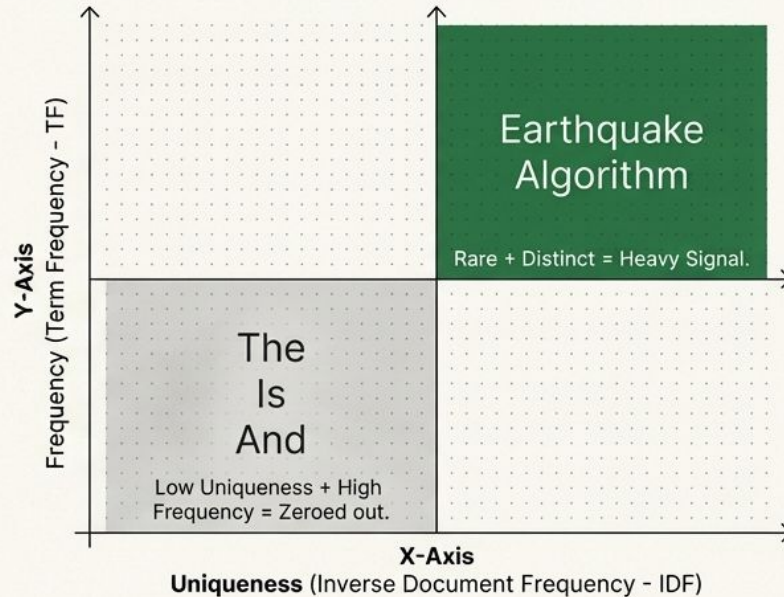
Document B: "AI likes me."

- Mathematical representation becomes identical
- Both documents have the same word frequency profile



This visual demonstrates how, in a Bag of Words model, different sentence structures can yield identical vector representations, as word order is disregarded.

In a massive sea of text, how can an algorithm figure out which words actually matter without knowing what any of them mean?



the cat sat
on the **mat**

Intuition: Not All Words Are Equal

Imagine reading 1,000 news articles.

- "The" appears everywhere → tells you nothing.
- "Earthquake" appears rarely → a strong signal!

TF-IDF is like a **smart highlighter**: it dims common words and brightens rare, meaningful ones.

- **TF** (Term Frequency): How often a word appears **in THIS document**.
- **IDF** (Inverse Document Frequency): How rare a word is **across ALL documents**.

TF × IDF = Importance

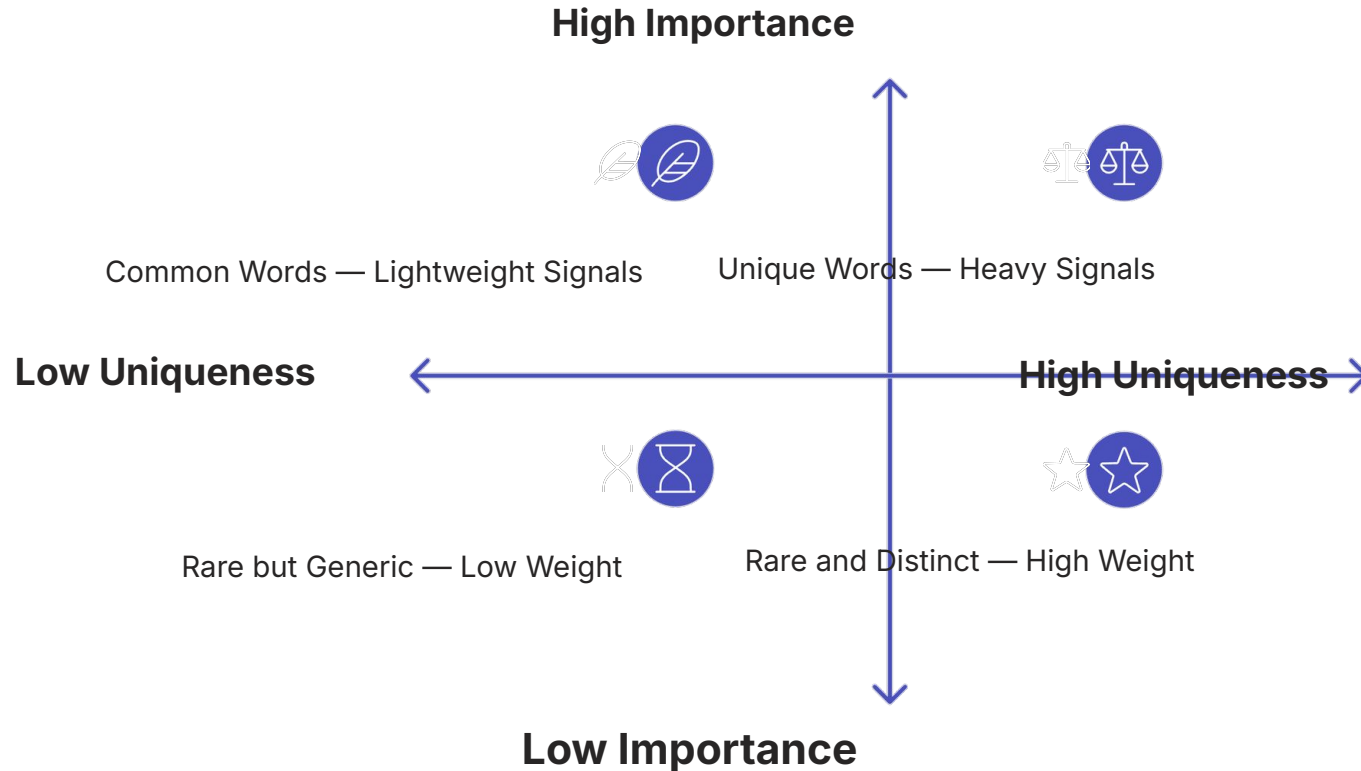
TF-IDF: Beyond Raw Counts

The Core Idea

- **TF-IDF** (Term Frequency–Inverse Document Frequency) reweights word counts to reward words that are frequent in a specific document but rare across the entire corpus. It is the NLP community's elegant solution to the "common word dominance" problem.
- **TF**: How often does this word appear in this document?
- **IDF**: How rare is this word across all documents?

$\text{TF-IDF} = \text{TF} \times \text{IDF} \rightarrow$ high score for document-specific terms

TF-IDF: Beyond Raw Counts



The TF-IDF Formula

- **TF** = Term Frequency in Document
- **IDF** = $\log(\text{Total Docs} / \text{Docs with Term})$
- **Score** = $\text{TF} \times \text{IDF}$
- Scores importance within a specific context

$\text{TF}(t, d)$ = frequency of term t in document d

$\text{IDF}(t, D) = \log\left(\frac{\text{Total Number of Documents (N)}}{\text{Number of Documents containing term } t}\right)$

$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t, D)$

$$\text{tf}(t, d)$$

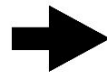
	blue	bright	can	see	shining	sky	sun	today
1	1/2	0	0	0	0	1/2	0	0
2	0	1/3	0	0	0	0	1/3	1/3
3	0	1/3	0	0	0	1/3	1/3	0
4	0	1/6	1/6	1/6	1/6	0	1/3	0

X

$$\text{idf}(t, D)$$

	blue	bright	can	see	shining	sky	sun	today
	0.602	0.125	0.602	0.602	0.602	0.301	0.125	0.602

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$



- TF-IDF: Multiply TF and IDF scores, use to rank importance of words within documents
- Most important word for each document is highlighted

	blue	bright	can	see	shining	sky	sun	today
1	0.301	0	0	0	0	0.151	0	0
2	0	0.0417	0	0	0	0	0.0417	0.201
3	0	0.0417	0	0	0	0.100	0.0417	0
4	0	0.0209	0.100	0.100	0.100	0	0.0417	0

Limitations of Count-Based Methods

No Word Order:

- "Dog bites man" and "Man bites dog" have identical BoW representations despite opposite meanings.

No Semantic Similarity:

- "King" and "Queen" are as unrelated as "King" and "table" in a BoW feature space.

Curse of Dimensionality:

- Vectors scale with vocabulary size. Real corpora produce vectors with tens of thousands of mostly-zero dimensions.

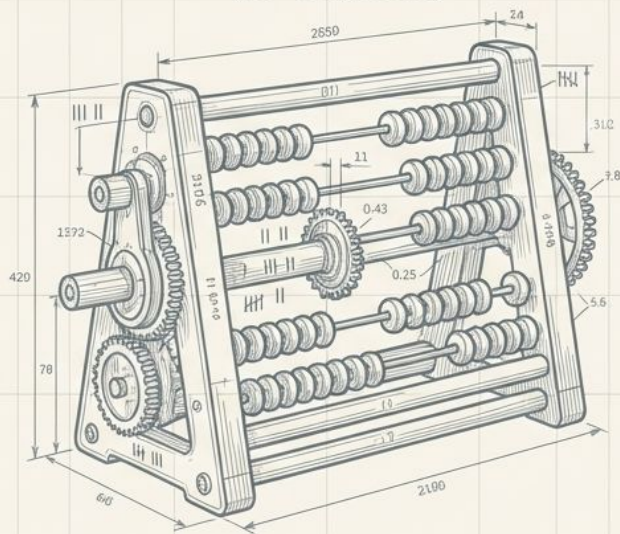
No Context Awareness:

- "Bank" (river) vs. "bank" (financial) are treated identically — polysemy is completely ignored.

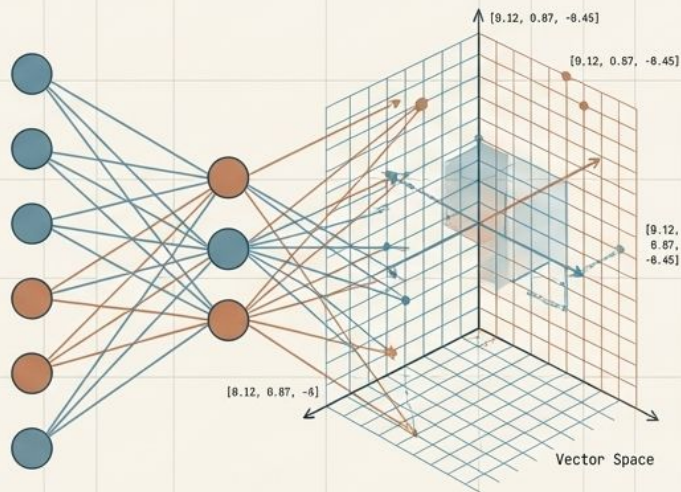
The Paradigm Shift: From counting to predicting

By 2013, NLP hit a wall with count-based sparse matrices. The breakthrough required abandoning counting entirely. What if we forced a neural network to predict missing words? This predictive pressure forces the model to compress meaning into compact, dense numerical vectors.

Count-Based



Predict-Based (Word2Vec, GloVe)



How does a child learn that coffee and tea are related without ever opening a dictionary?



I drink [_____] in the morning.



Learning by Predicting:
Meaning is defined by neighboring words.
Context forces the model to cluster similar concepts together.

Prediction Based Methods

The Distributional Hypothesis: Words are defined by their company

Children do not learn language by reading dictionaries; they learn by observing context. By forcing a model to predict the blank, we inherently force it to map the geometric relationships between concepts.

The barista poured hot _____ into the mug.



Shared context creates
shared geometry.

Learning by Predicting

Think about how a child learns language. They don't memorize a dictionary — they learn from **context**.

Hearing "I drink ____" hundreds of times, they learn "coffee," "water," "juice" all belong there.

Word2Vec does the same: it learns word meaning by predicting what words appear near each other.

- The key insight: instead of **counting**, we now **predict**.
- This prediction forces the model to learn deep semantic relationships.

Prediction-Based Embeddings: The Revolution

- 1 Transition from word counting to context prediction.
- 2 Generation of compact, dense numerical vectors.
- 3 Capture of complex semantic relationships.

Introduction to Word2Vec

Proposed by Google in 2013. Transitions from counting to predicting — creating dense, low-dimensional vectors.

The Distributional Hypothesis

Meaning is defined by neighboring words. Context reveals semantic relationships. Foundation of modern word embeddings.

CBOW

Faster training, ideal for frequent words. Predicts target word from surrounding context.

Skip-gram

Slower but better for rare words. Predicts surrounding context from a single word.

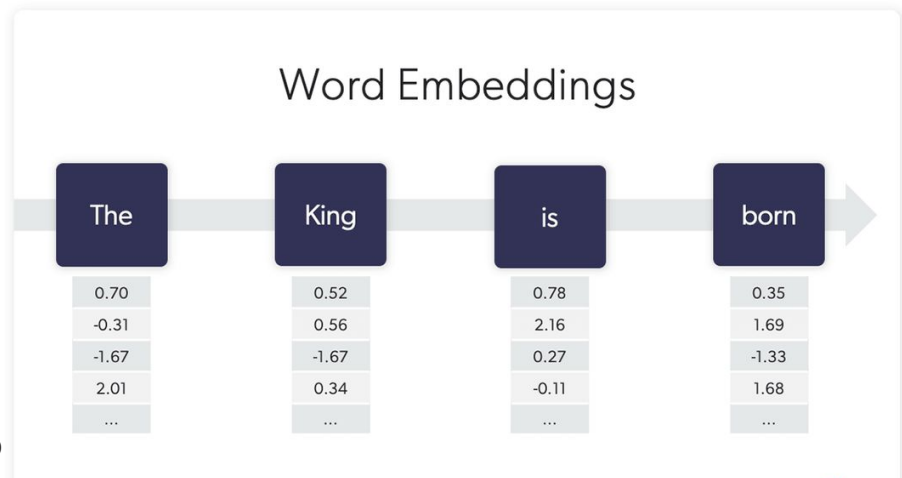
Introduction to Word2Vec

Word2Vec is a technique that transforms words into numerical vectors using their predictive power—enabling models to reason about language with striking accuracy.

The classic example:

King – Man + Woman = Queen.

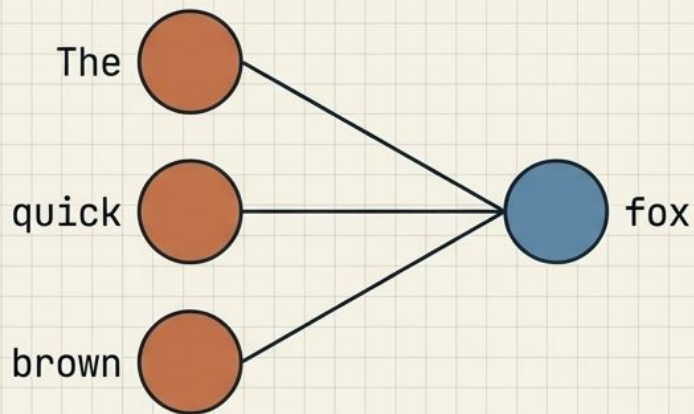
This is learned geometry. Word2Vec maps words into a high-dimensional space where meaning, relationships, and analogies emerge naturally from the data.



Word2Vec Architectures: Two ways to read the neighborhood

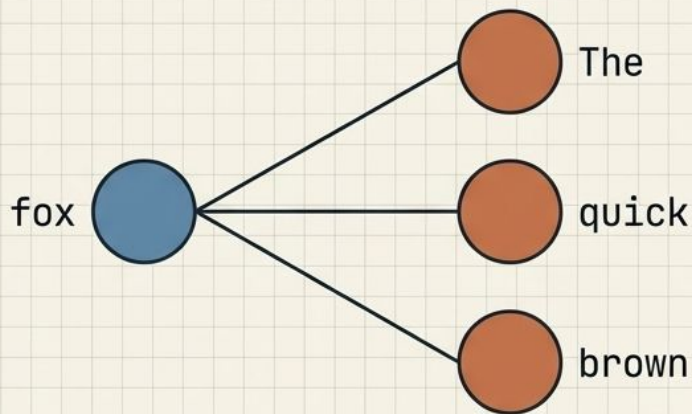
Word2Vec relies on two inverse neural network architectures to generate its dense vectors.

CBOW (Continuous Bag of Words)



Predicts center word from context.
Faster, best for frequent words.

Skip-gram



Predicts context from center word.
Slower, best for rare words.

Intuition: Fill in the Blank

Remember fill-in-the-blank exercises from school? "The cat sat on the ____."

CBOW is exactly this: given the **context words** around a gap, predict the missing **target word**.

The model reads the neighborhood, averages what it sees, and makes its best guess. Over millions of examples, it learns that words used in similar contexts must mean similar things.

Architecture 1: CBOW

CBOW learns representations by predicting a target word from context.

Data Generation: The Sliding Window

Window 1

["The", "quick"] → "brown"

Window 2

["quick", "brown"] → "fox"

Window 3

["brown", "fox"] → "jumps"

CBOW: Architecture Breakdown

Input Layer

One-hot vectors for each context word (size = $|V|$). Multiple context words are fed simultaneously.

Hidden Layer (Projection)

Computes the average (or sum) of all context word embeddings. Crucially, there is NO activation function here — it's a pure linear projection.

Output Layer (Softmax)

Applies Softmax over the entire vocabulary to produce a probability distribution. Softmax is required to convert raw scores into valid probabilities that sum to 1, so we can identify the most likely target word.

CBOW: Learning & The Softmax Bottleneck

Learning Mechanism

Backpropagation pulls target word embeddings toward the averaged context vector.

Words with similar contexts converge in vector space, encoding semantic similarity.



Dual Matrix Reality: W_{input} stores word embeddings; W_{output} is discarded after training.

Softmax Bottleneck

- Computing Softmax over large vocabularies is computationally prohibitive.
- Negative Sampling approximates the result to enable scalable training.

An Example

There lived a king called Ashoka in India. After Kalinga battle, he converted to Buddhism. This mighty **king ordered his** ministers to put together a peaceful treaty with their neighboring kingdoms. The **emperor ordered his** ministers to also build stupa, a monument with Buddha's teachings

Training samples

lived, a → There

a, king → lived

ordered, his → king

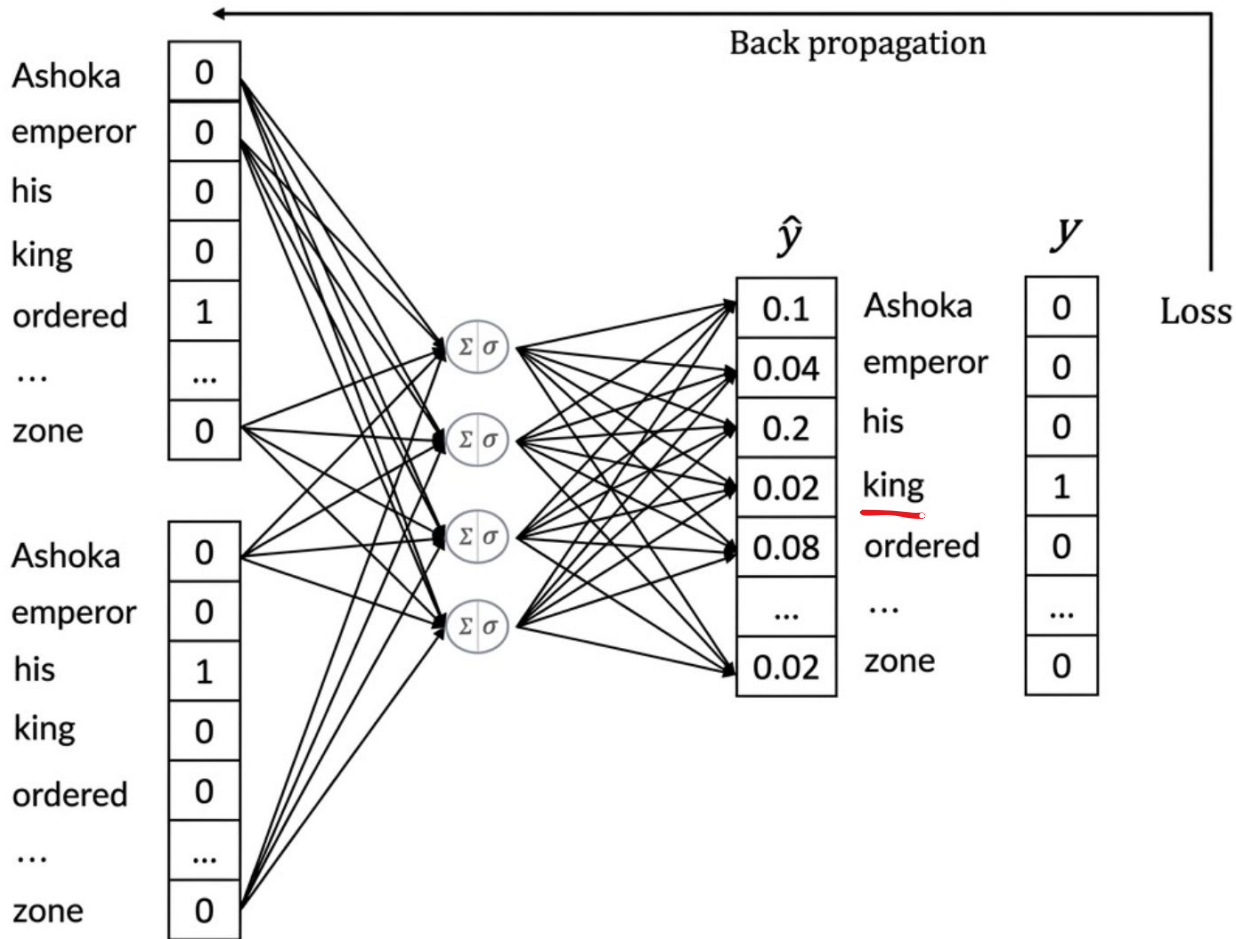
ordered, his → emperor

CBOW

king ordered his

ordered

his



Intuition: Thinking in Reverse

CBOW asked: "Given the neighborhood, what's the center word?"

Skip-gram flips this: Given **one word**, predict what words surround it.

Imagine a detective finds a clue like "volcano." They'd predict related words: "eruption," "lava," "Hawaii."

This reverse approach makes Skip-gram exceptionally powerful for understanding **rare words** and their unique contexts.

Architecture 2: Skip-gram

- Inverse of CBOW architecture
- Predicts context words from a single target word
- Example: Input "Cat" predicts [the, is, meowing]

Training Optimization: Negative Sampling

- Avoids updating millions of weights simultaneously
- Updates only the target word and a small set of random "negative" words
- Makes training on large vocabularies feasible

The Magic of Dense Vectors

- Dimensions reduced from 50k+ to ~300
- Information is "Distributed" across all values
- Every dimension represents a latent feature

Long Sparse Vector

[0, 0, 1, 0, 0, ... 0]

Short Dense Vector

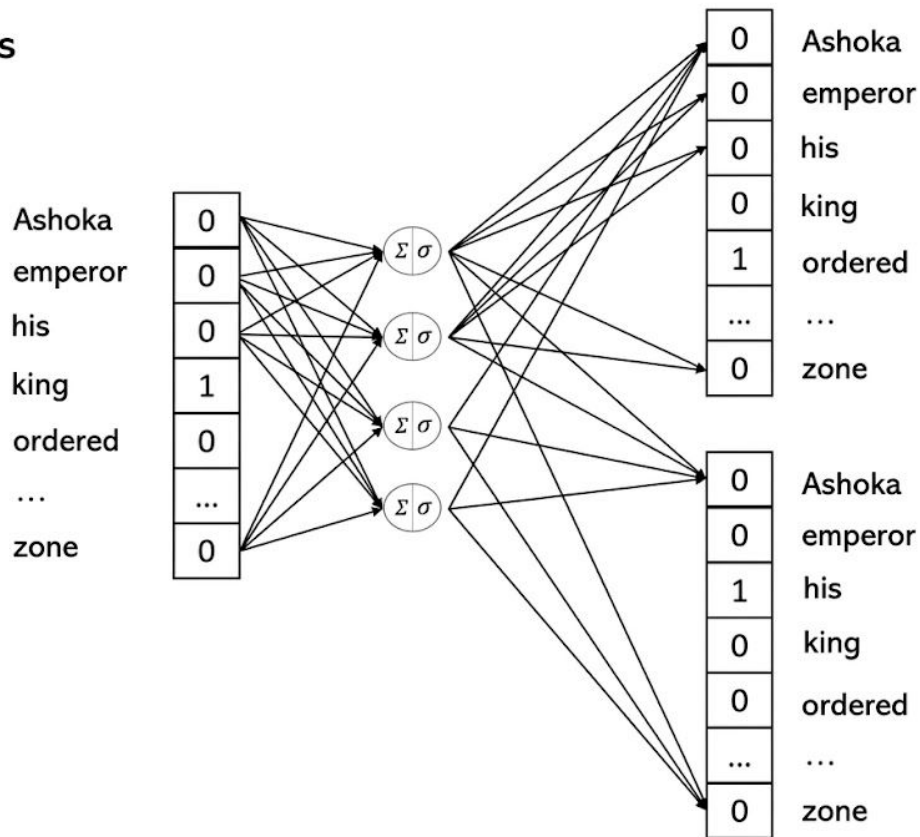
[0.2, -0.5, 1.8, ..., 0.7]

Vector Arithmetic: Semantic Logic

- The 'King - Man + Woman = Queen' proof
- Proves vectors capture gender and royalty
- Geometric relationships mirror human logic

Skip Gram

king ordered his



Intuition: The Best of Both Worlds

Word2Vec learns word relationships by analyzing local contexts, sentence by sentence.

GloVe takes a different approach: it first reads the **entire corpus** to build a global co-occurrence matrix.

Then, it leverages this global statistical information to learn word vectors, combining the strength of both "counting" and "prediction" methods.

To understand the world, is it better to learn from your immediate conversations, or to step back and study the entire library at once?



Local Context (Predict-Based)

Captures nuanced semantic relationships from immediate sliding windows.

The Hybrid Approach:

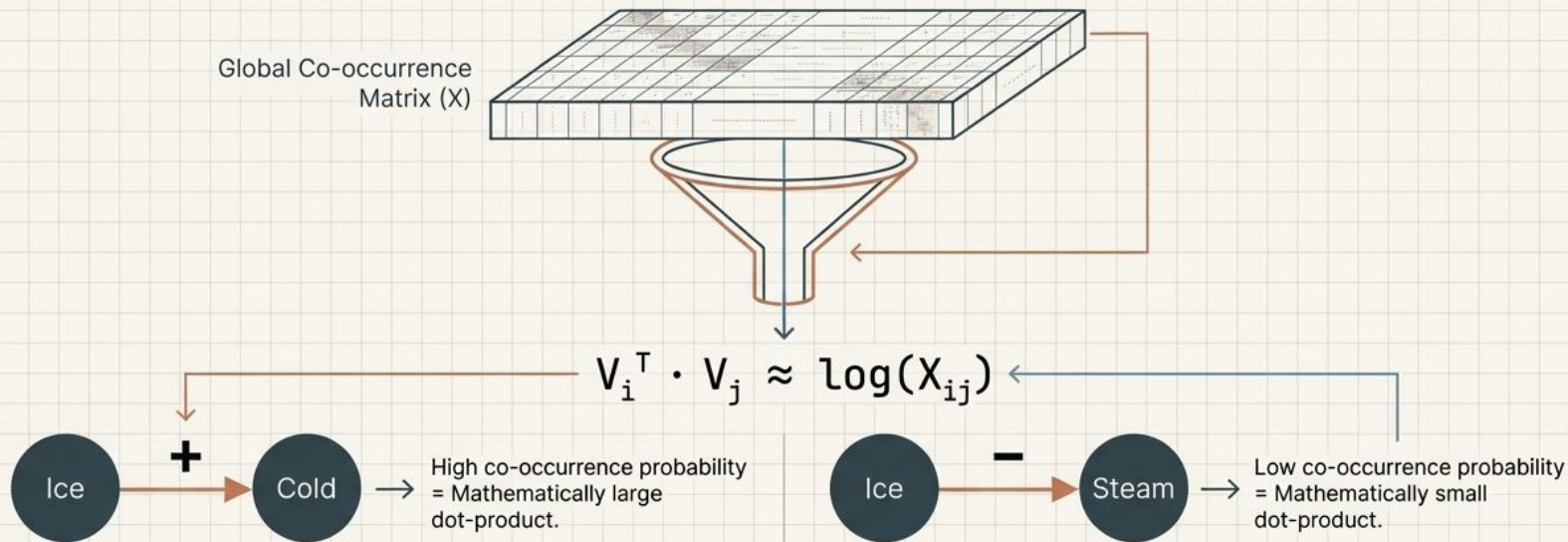
The Hybrid Approach: Training vectors to predict global co-occurrence directly.

Global Statistics (Count-Based)

Efficiently decomposes entire corpus-wide co-occurrence probabilities.

GloVe: Bridging global statistics with local prediction

Word2Vec learns by reading a corpus linearly. GloVe (Global Vectors) takes a hybrid approach: it constructs a massive global co-occurrence matrix, then trains vectors to predict those global probabilities.

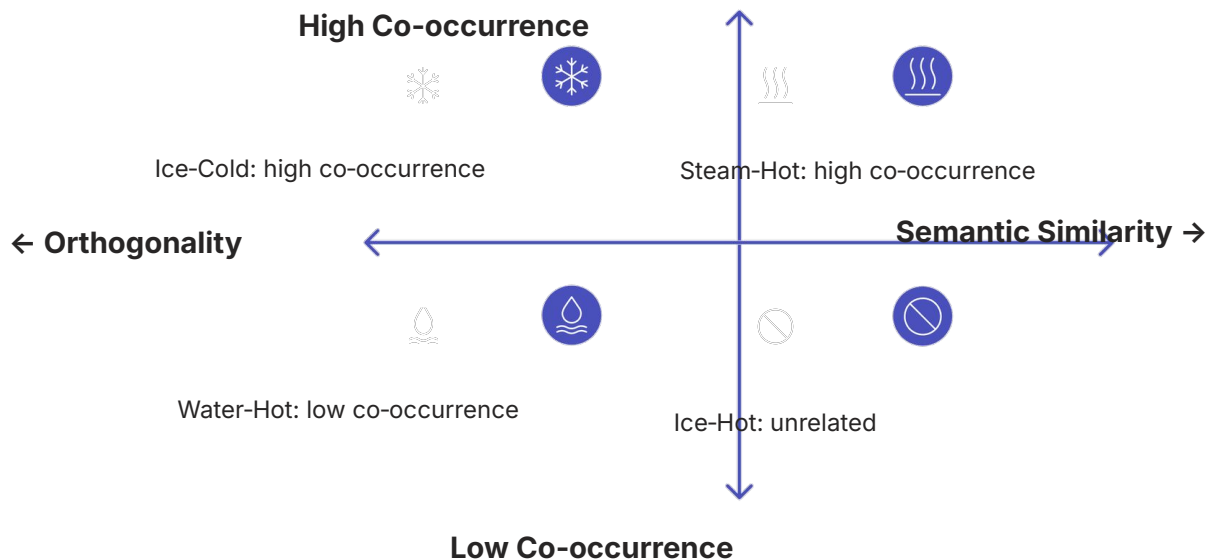


GloVe: Bridging Count-Based & Predict-Based Models

- **Count-Based Methods:** Efficiently decompose global co-occurrence matrices for corpus-wide statistics.
- **Predict-Based Models:** Effectively capture nuanced semantic relationships from local context windows.
- **GloVe's Hybrid Approach:** Unifies these methods by training vectors to predict global co-occurrence probabilities.

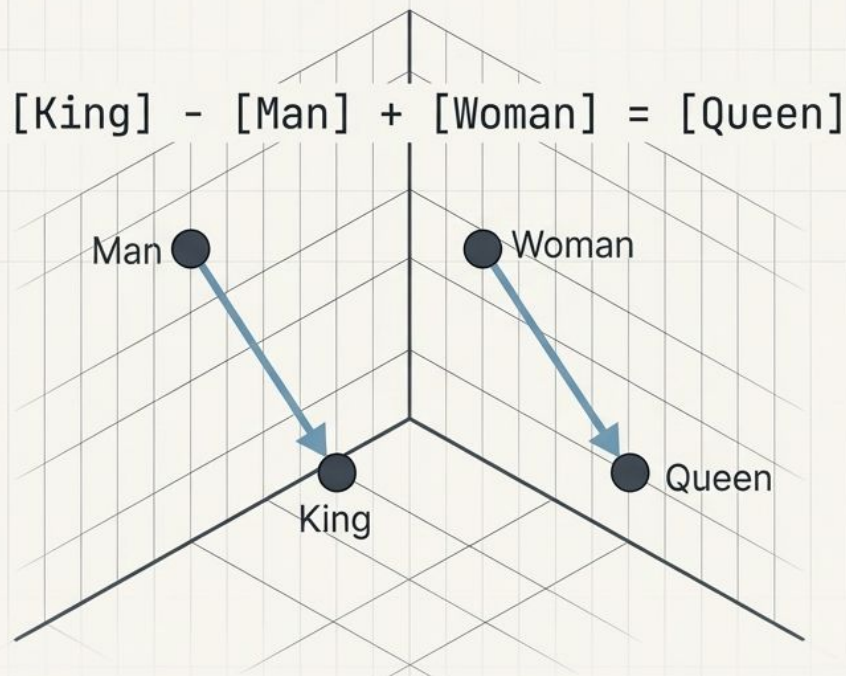
GloVe: Global Co-occurrence Matrix (X_{ij})

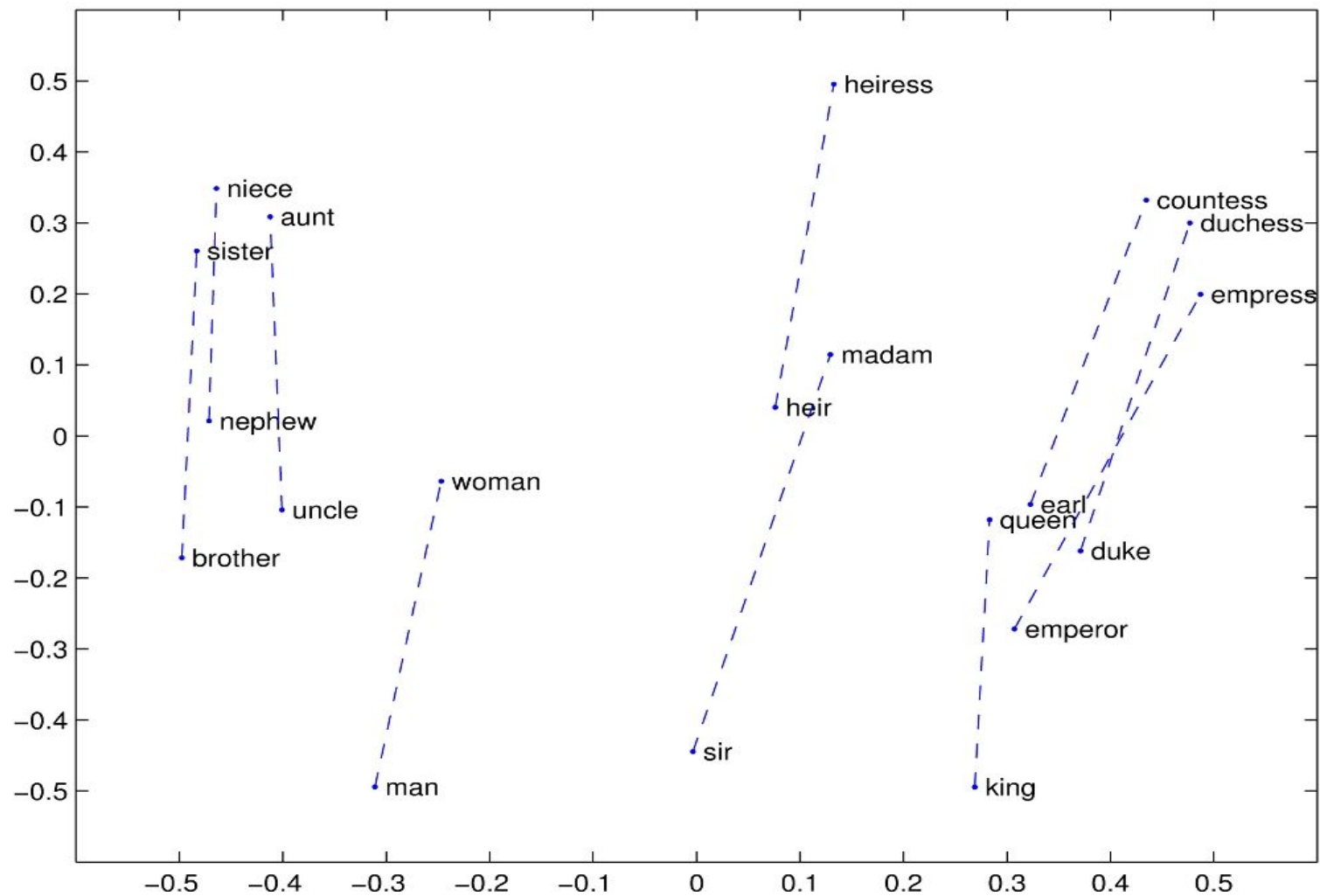
- X_{ij} counts how many times word j appears in the context of word i across the entire corpus.
- Unlike window-based models, GloVe aggregates global statistics before the learning process begins.
- This ensures vectors capture corpus-wide patterns rather than relying on local word proximity.



Semantic Arithmetic: Proof that meaning is geometry

The ultimate validation of dense embeddings is that human logic translates flawlessly into algebraic operations. By capturing latent features across hundreds of dimensions, the model can literally add and subtract concepts.





GloVe: The Core Objective

The mathematical desire:

- GloVe wants the **dot product of two word vectors** to be faithful to their **co-occurrence probability**
- Formally: $V_i^T \cdot V_j \approx \log P(j|i)$

Why log probability?

- $P(j|i)$ = probability that word j appears in the context of word i
- Taking the **log** converts multiplication into addition — mathematically convenient and numerically stable
- The dot product $V_i^T \cdot V_j$ measures **vector similarity** — high for related words, low for unrelated ones

The semantic power:

- If "ice" and "cold" frequently co-occur $\rightarrow P(\text{cold}|\text{ice})$ is high $\rightarrow$ their dot product should be **large**
- If "ice" and "hot" rarely co-occur $\rightarrow P(\text{hot}|\text{ice})$ is low $\rightarrow$ their dot product should be **small**
- This encodes **semantic relationships directly into the geometry** of the vector space

$$V_i^T \cdot V_j \approx \log(X_{ij})$$

Recap: The Predictive Jump

- Vectors represent semantic meaning
- Dimensionality provides efficiency
- Context drives learning performance

Evaluating Embeddings:

- Intrinsic: Word similarity and analogy tasks
- Extrinsic: Downstream task performance (Sentiment, NER)
- Visualization: t-SNE cluster inspection

Managing Vector Size & Visualization

- Accuracy vs. computation time trade-off
- Standard dimensions: 100, 300, or 768
- Compression for mobile edge devices

Visualizing with PCA/T-SNE:

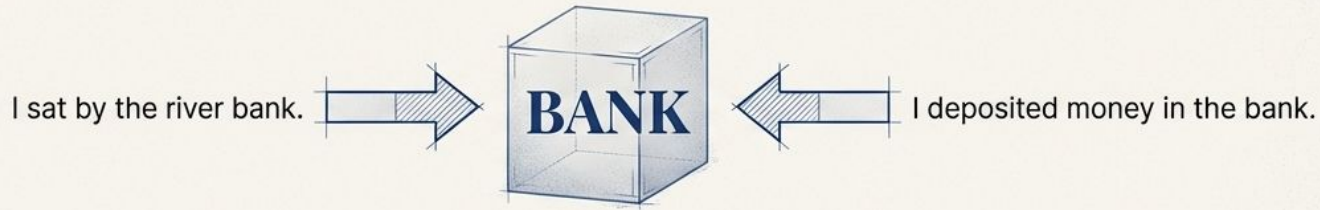
- Projects high-dimensional vectors onto 2D/3D space
- Reveals synonym clusters and semantic neighborhoods
- Tools: t-SNE, PCA, UMAP

Section Recap:

- Embeddings are universal across data types
- The same math applies to DNA, text, and sound

If the word 'bank' means both a river's edge and a financial vault, why is giving it a single, static mathematical coordinate fundamentally flawed?

Static Word Embeddings



Contextual Embeddings (BERT/GPT)



The Contextual Revolution: Meaning is not static. Modern language models generate an entirely new, distinct vector for a word in real-time based on the surrounding sentence.

The Polysemy Problem: Meaning depends on the moment

Static embeddings assign a single, rigid vector to a word regardless of how it is used. Contextual embeddings shift the paradigm by reading the entire sentence dynamically to generate a unique vector based on live surroundings.

Static Embeddings (e.g., Word2Vec)

The river bank was muddy

Bank of America

Vector $[0.2, -0.5, 0.8]$

Same vector for all contexts.

Contextualized Embeddings (e.g., BERT)

The river bank was muddy

Bank of America

Vector $[-0.3, 0.6, 0.4]$

Vector $[0.7, -0.1, 0.5]$

Different vectors based on context.

Dynamic Embeddings

Intuition: Words Change Meaning with Context

The word "**bank**" has different meanings depending on its context:

- "**river bank**" (land beside water)
- "**bank account**" (financial institution)

Static embeddings (like Word2Vec) assign "bank" one fixed vector, regardless of its usage.

Contextual embeddings (BERT, GPT) generate a DIFFERENT vector for "bank" for every sentence it appears in.

This is like the difference between a **photo ID** (static) and a **live video call** (dynamic). Models like BERT read the **entire sentence bidirectionally** to understand a word's true meaning.

The Next Frontier: Contextual Embeddings

From static vectors to dynamic, context-aware representations.

The Shift to Contextual Embeddings

- **Static (Word2Vec):** "Bank" has one vector
- **Contextual (BERT/GPT):** "Bank" vector changes based on sentence
- **The evolution continues toward true understanding**

Static vs Contextualized Embeddings

Static Embeddings

"I went to the bank"

Vector [0.2, -0.5, 0.8, ...]

"The river bank was muddy"

Vector [0.2, -0.5, 0.8, ...]

"Bank of America"

Vector [0.2, -0.5, 0.8, ...]

Same vector for all contexts

Contextualized Embeddings

"I went to the bank"

Vector [0.8, 0.2, -0.3, ...]

"The river bank was muddy"

Vector [-0.3, 0.6, 0.4, ...]

"Bank of America"

Vector [0.7, -0.1, 0.5, ...]

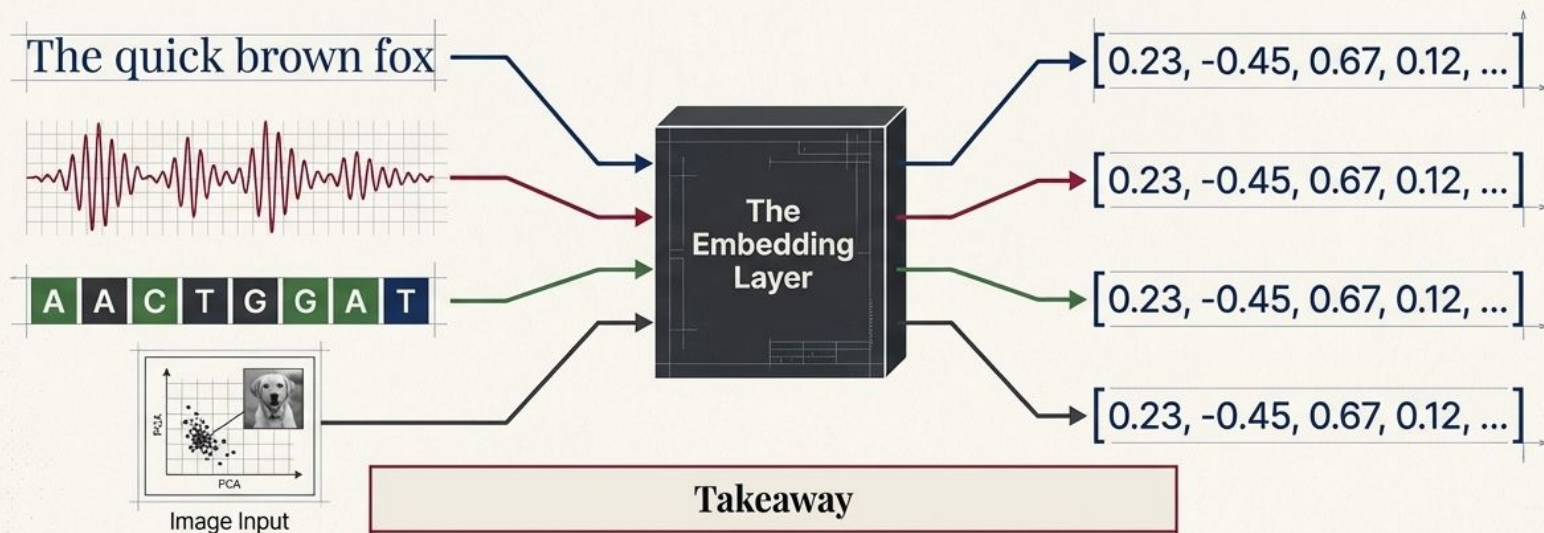
Different vectors based on context

The Vector Evolution Matrix

A definitive comparison of how language representation evolved from isolated integers to dynamic dimensional spaces.

	Captures Word Order?	Captures Semantic Meaning?	Handles Polysemy?	Dimensionality
One-Hot Encoding	✗	✗	✗	Massive (Sparse)
Bag of Words	✗	✗	✗	Massive (Sparse)
TF-IDF	✗	✗	✗	Massive (Sparse)
Word2Vec / GloVe	✗	✓	✗	Compact (Dense)
Contextual (BERT)	✓	✓	✓	Compact (Dense)

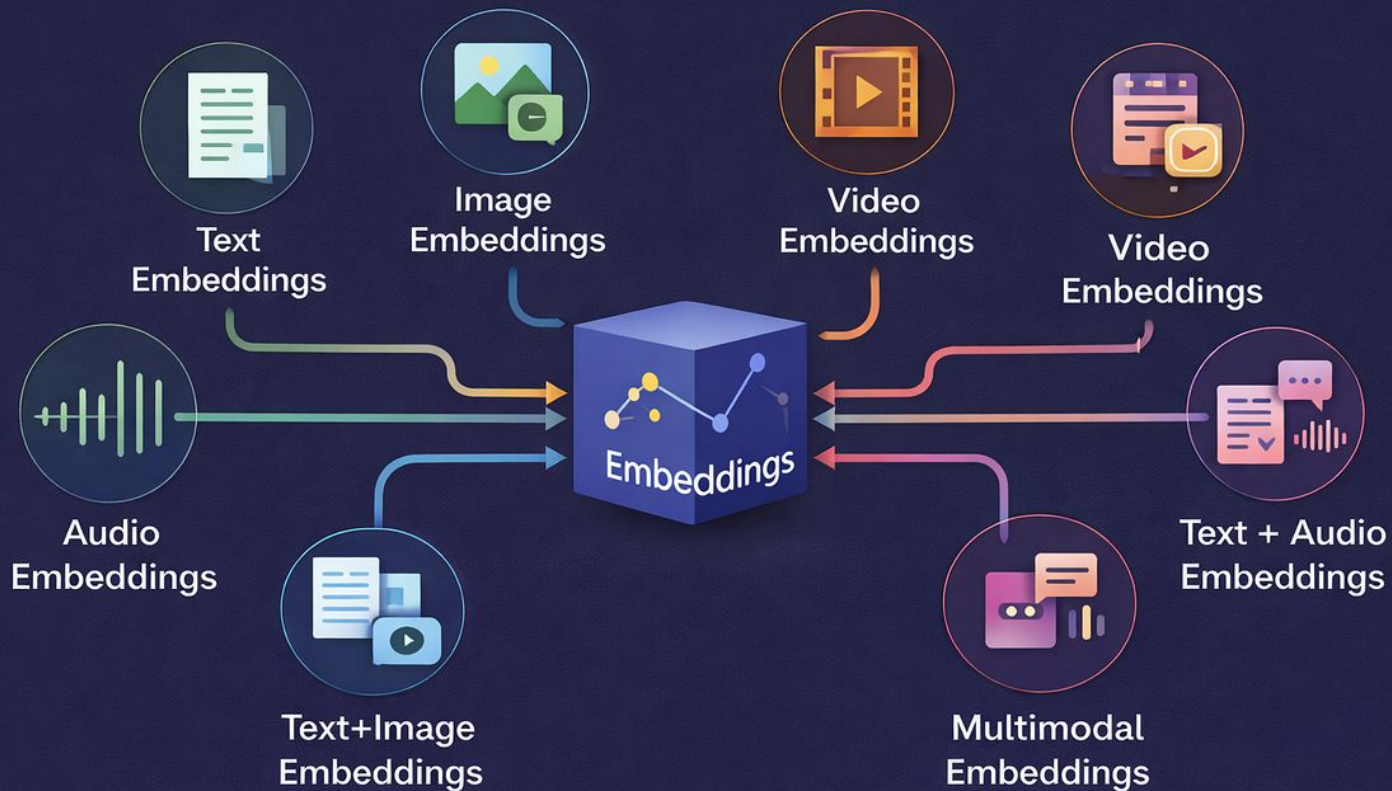
If meaning is just a coordinate in a mathematical space, could a DNA sequence, a Beethoven symphony, and a photograph all exist on the exact same map?



Takeaway

The Universal Language of AI: Any sequence of symbols can be mapped into a continuous vector space where similarity equals proximity.

7 Types of **Embeddings** That Power Modern **Semantic Search**: A Complete Guide



Cross-Domain Applications: Beyond Text

What if the same mathematical trick that understands "king" could also understand a DNA sequence, a musical note, or a pixel? Embeddings are a **universal language** — any sequence of symbols can be mapped into a vector space where **similarity = proximity**. Think of it like GPS coordinates: every location gets a coordinate. Embeddings give every "thing" — word, gene, sound — its own coordinate in meaning-space.



Text

Semantics and sentiment analysis



Image

Visual search and similarity



Audio

Speech recognition and patterns



DNA

Genomic sequence classification

Text Applications

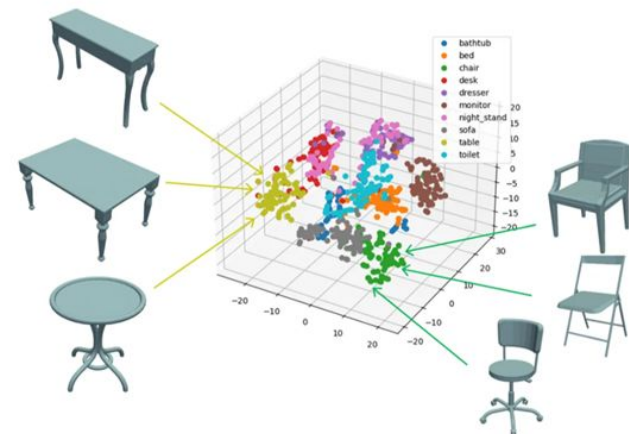
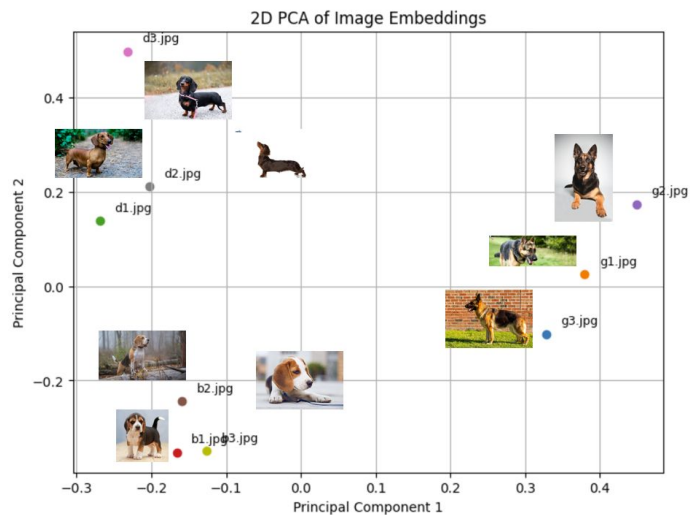
- Sentiment Analysis using positive and negative clusters
- Machine Translation by mapping languages to one space
- Semantic Search for finding meaning beyond keywords

Audio & Image Embeddings

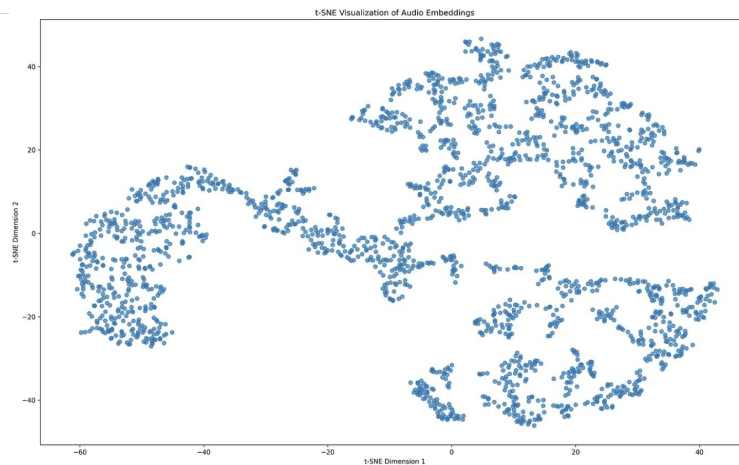
- Convert sound waves to MFCC vectors
 - Use Wav2Vec for slice prediction
 - Power speech-to-text and recommendation
-

Image Embeddings

- Pixels → CNN Features → Concept Vectors
- Used in Facial Recognition and Google Lens
- Enables visual search and similarity matching



Object retrieval in embedding space: similar objects are expected to have a small embedding distance.



When you apply embeddings to an audio library, you can explore implicit structures such as similar songs

DNA & Bio-Embeddings

- Nucleotides treated as letters
- Gene sequences treated as sentences
- DNABERT decodes the "Language of Life"

Why It Matters:

- Predicting gene mutations and disease
- Identifying similar protein structures
- Accelerating drug discovery

A A C T G G A T G A T C



extracted 8-mer

A A C T G G A T

the vector for 8-mer

[0.23, 0.45, 0.67, ..., 0.78]

A C T G G A T G

[0.32, 0.55, 0.17, ..., 0.08]

C T G G A T G A

[0.32, 0.54, 0.73, ..., 0.38]

T G G A T G A T

[0.33, 0.41, 0.17, ..., 0.14]

G G A T G A T C

[0.13, 0.35, 0.42, ..., 0.41]



normalization

[0.26, 0.46, 0.43, ..., 0.36]

Key Takeaways & Conclusion

Math is Meaning:

[Foundations of Vector Embeddings:
A Comprehensive Study Guide](#)

- Semantics are now measurable coordinates.
- Relationships are preserved.
- Geometry encodes human concepts.

Cross-Domain Unity:

- Unified math powers DNA, text, and sound
- AI consolidates diverse fields via vectors
- Embeddings are the universal language of AI